



REDLINE DAYLIGHT • AUTOMATION HANDBOOK

الدليل الشامل لتعلم Ansible

شرح عملي ومنظم لـ **Ansible** مع الحفاظ على الـ **English Technical Terms** كما تُستخدم فعليًا في بيئة الـ DevOps — من الـ Architecture والـ SSH Keys، إلى الـ Modules والـ Inventory والـ Ad-hoc Commands والـ Playbooks، وصولًا إلى الـ Roles والـ Templates والـ Handlers و**Ansible Galaxy** — مع الصورة الكاملة والـ Mental Model للحفظ.

By **Nabawy**

AUTHOR & COMPILER • إعداد وتنسيق

Roles

 Galaxy +

Playbooks

YAML

SSH

اتصال آمن

REDLINE EDITION

Daylight • A4-Fixed

المحتويات

16	 المثلث Hierarchy والـ Playbooks	13	3	 ملخص تنفيذي	01
17	 تشغيل الـ Playbook + فهم النتائج	b•13	4	 Intro — مقدمة والفكرة الأساسية	02
18	 Roles — الفكرة ولماذا نستخدمها	14	5	 Features 1/2 — Agentless	03
19	 Role Structure 1/2 — tasks & handlers	15	6	 Features 2/2 — Declarative + Idempotency	04
20	 Role Structure 2/2 — vars..meta	b•15	7	 Architecture — المعمارية	05
21	 استخدام Role + منصة Galaxy	16	8	 Comparison — مقارنة طرق الإدارة	06
22	 Role vs Galaxy vs ansible-galaxy	17	9	 Use Cases — مجالات الاستخدام	07
23	 الصورة الكاملة لـ Ansible	18	10	 Installation + VS Code	08
24	 Mental Model + Final Chain	🧠	11	 SSH Keys — الاتصال الآمن	09
25	 Quick Revision Table + الخلاصة	📌	12	 Modules — وحدات التنفيذ	10
			13	 Inventory — ملف الـ Servers	11
			14	 Ad-hoc Commands	12
			15	 YAML Basics — أساسيات الصيغة	

TASK [دليل ألوان الحالة] status legend

failed — حدث Error في الـ Task changed — تم تنفيذ تغيير بنجاح ok — الـ Server بالحالة المطلوبة، لا تغيير RED — عناوين وهيكل (ديكور بنيوي)

[DEBUG] كيف تستخدم الكتاب: 18 فصلاً مترابطاً يغطي كل الـ Concepts — وكل المصطلحات التقنية بالإنجليزية كما تُستخدم فعلياً في بيئة الـ DevOps. الأكواد تُقرأ من اليسار والنص من اليمين. الفصول 14-17 مخصصة كاملة للـ Roles والـ Galaxy لأنها أهم نقطة توسع بعد الـ Playbooks.

الملخص التنفيذي — الطريق كاملاً

قبل لمس أي Command، هذه القصة كاملة — كل جملة رابط يأخذك لفصلها.

نبدأ في **الفصل 1 — Intro: Ansible** أداة **Automation** مفتوحة المصدر في ثلاثة مجالات — **Configuration Management** و**Infrastructure Provisioning** و**Network Automation** — وفكرتها الجوهرية أنك تحدد الـ **Desired State** وAnsible يتولى التنفيذ. ثم مميزات الأربع: **Agentless** (SSH/WinRM بلا Agents)، «Apache» — **Simple Syntax + Declarative + Idempotency** يجب أن يكون موجوداً لا «نقذ apt install».

في **الفصل 4 — Architecture: Control Node** عليه Ansible وPlaybooks وInventory يتصل بـ **Managed Nodes** عبر SSH/WinRM. وفي **الفصل 5 — Comparison** نقارن Manual وBash وAnsible على 3 Servers لتثبيت Nginx. وفي **الفصل 6 — Use Cases** نفصل بين Ansible (Configuration) وTerraform (Provisioning).

العددة العملية: **التثبيت + VS Code Extensions**، ثم Private Key — **SSH Passwordless** يبقى، Public Key يُرفع. ثم **الـ Modules** **السبعة**، **الـ Inventory** بالمجموعات، و**الـ Ad-hoc Commands** الفورية.

اللغة والتنفيذ: **YAML** — **الـ Indentation** والهياكل الثلاثة — ثم **الـ Playbook** و**الـ Hierarchy** الـ Play→Task، وتشغيله وقراءة **الـ ok/changed/failed**. وعندما يكبر المشروع: **الـ Roles** (Build Once, Reuse Everywhere)، **هيكل المجلدات السبعة**، **استخدام Role + منصة Galaxy**، والفرق الجوهرى بين **Role وGalaxy وansible-galaxy**.

نختم بالصورة الكاملة في **الفصل 18**، ثم **الـ Mental Model للحفظ** و**جدول الـ Quick Revision**. الطريق واضح. ►

KEY IDEA الفكرة الجامعة: أنت تحدد الـ **Desired State** — وAnsible يتولى تنفيذ الخطوات المطلوبة للوصول إليها، دون تغييرات غير ضرورية عند التكرار.

Intro مقدمة ونظرة عامة

Ansible هي أداة Automation مفتوحة المصدر (Open-Source) تُستخدم بشكل أساسي في ثلاثة مجالات:

Network Automation

أتمتة أجهزة الشبكة.

Infrastructure Provisioning

بناء وتجهيز البنية التحتية.

Configuration Management

إدارة الإعدادات داخل الـ Servers.

KEY IDEA • الفكرة الأساسية [أنت تحدد الـ **Desired State** التي تريد أن يصل إليها الـ Infrastructure — و**Ansible** يتولى تنفيذ الخطوات المطلوبة للوصول إليها. أنت توصف النهاية، وهو يصنع الطريق.

TASK [كيف تعمل الفكرة عملياً؟] the core loop



نفس الـ Code في كل مرة = نفس النتيجة. لا Scripts عشوائية ولا خطوات منسية.

DEBUG • لماذا هذا مهم؟ [في الإدارة اليدوية، كل Server حالة مستقلة: من لَمْس ماذا؟ ما الذي تغيّر؟ مع Ansible، الـ Infrastructure كلها موصوفة في **Code** واحد قابل للقراءة والمراجعة وإعادة التنفيذ — هذا جوهر الـ IaC mindset الذي تبنيه Ansible.

EXAM احفظ المجالات الثلاثة بالترتيب: **Configuration Management • Infrastructure Provisioning • Network Automation** والجملة الجوهرية: «تحديد الـ Desired State و**Ansible** ينقذ الخطوات».

أربع مميزات تجعل **Ansible** بهذا الشكل — أولها وأشهرها: **الفصل 2: Agentless**

Agentless بدون عميل —

من أهم مميزات Ansible أنها **Agentless** — يعني لا تحتاج إلى تثبيت برنامج أو **Agent** خاص بـ **Ansible** على الـ Managed Nodes. لا تثبيت مسبق، لا صيانة Agents، لا Versions تتزامن.

في Windows 🖥️

يمكن استخدام **WinRM** أو **SSH** للاتصال والتنفيذ.

في Linux 🐧

الاتصال عادةً باستخدام **SSH**.

المتطلب الأساسي على الـ Managed Node: وجود بيئة **Python**.

◀ TASK [القناة بين الجهازين] the connection

Managed Node

Ubuntu / CentOS / DB

► — SSH / WinRM — ◀

Control Node

عليه Ansible فقط

الـ Control Node يتصل عند الحاجة وينفذ ويرحل — الـ Managed Node لم يتغير إعداداته أصلاً.

[KEY IDEA] قارن مع أدوات أخرى تطلب Agent على كل جهاز: تثبيت، تحديث، صيانة. في Ansible: المتطلب = **SSH + Python** (موجود افتراضياً في أغلب التوزيعات). هذا يقلل الـ friction في الـ Onboarding لأدنى حد.

[WATCH OUT] «Agentless» لا تعني «بلا متطلبات» — الـ Python environment على الـ Managed Node شرط أساسي في Linux. أنظمة نادرة بلا Python تحتاج خطوة تجهيز واحدة قبل أول Run.

EXAM 🎓 سؤال كلاسيكي: «ما معنى Agentless وما المتطلبات؟» → لا **Ansible Agent** يُثبَّت على الـ **Managed Nodes** · الاتصال عبر **SSH** (Linux) أو **WinRM** (Windows) · المتطلب: **Python** على الـ Target.

🔗 الميزة الثانية: لغة الكتابة نفسها — لماذا YAML؟ → **الفصل 3: Simple Syntax + Declarative**

Declarative + Idempotency

2.2 Simple Syntax

ال Playbooks تُكتب باستخدام **YAML** — مصممة بحيث تكون سهلة القراءة والكتابة، وتعتمد بشكل أساسي على ال **Indentation** لتنظيم ال structure. (التفصيل الكامل في فصل YAML لاحقًا).

2.3 Declarative Approach

في ال Declarative Approach أنت تحدد ما الذي تريده في النهاية بدلًا من كتابة كل خطوة يدوية للوصول إليه:

```
- name: Install Apache
  apt:
    name: apache2
    state: present
```

✓ أنت قلت

Apache must exist

وحدة هدفية — `state: present` = Desired State

✗ أنت لم تقل

apt install

أمر تنفيذي أعمى — ينفذ دائمًا حتى لو غير مطلوب.

Ansible يحدد بنفسه ما إذا كان يحتاج إلى إجراء تغيير للوصول إلى هذه الحالة — أم أن كل شيء على ما يرام.

[**DEBUG**] Ansible يدعم أيضًا ال **Imperative Approach** عند استخدام ال **Ad-hoc Commands** — أمر فوري لمهمة سريعة. الأسلوبان معًا: Playbooks للتوصيف الدائم، Ad-hoc للتنفيذ اللحظي.

2.4 Idempotency — التكرار الآمن

ال Idempotency تعني أن تنفيذ نفس ال Automation عدة مرات لا يؤدي إلى تغييرات غير ضرورية:

Second Run 🎬

```
Apache موجود بالفعل
↓
No change required
↓
ok
```

First Run 🎬

```
غير مثبت Apache
↓
Ansible installs Apache
↓
changed
```

🎓 **EXAM** لو كان Apache مثبتًا بالفعل، Ansible لن يعيد تثبيته بدون حاجة — يظهر ok لا changed. وفرق Declarative/Imperative: الأول يوصف الحالة (Playbooks)، الثاني ينفذ أمرًا فوريًا (Ad-hoc).

🔗 المميزات اكتملت — الآن الصورة المعمارية: **الفصل 4: Architecture**

Ansible Architecture معمارية —

تتكون الـ Ansible Architecture بشكل أساسي من مكونين: **Control Node** و **Managed Nodes**.

[Architecture - المكونان وما يحملانه]



الـ Control Node يحمل كل العتاد الفكري — الـ Managed Nodes يستقبلون وينفذون فقط.

3.1 Control Node — عقدة التحكم

هو الجهاز أو الـ Server الذي يتم تثبيت **Ansible** عليه. من خلاله يقوم الـ DevOps Engineer بـ:

- ▶ تشغيل أوامر **Ansible**.
- ▶ تشغيل الـ **Playbooks**.
- ▶ إدارة الـ **Managed Nodes**.

يمكن أن يكون: **WSL Ubuntu Environment** · Linux Server · Laptop — أي جهاز تستطيع عليه تشغيل **Ansible**.

3.2 Managed Nodes — العقد المُدارة

هي الـ **Target Machines** التي سيقوم **Ansible** بإدارتها. تستقبل الـ Instructions من الـ Control Node وتنفذها بدون الحاجة إلى تثبيت **Ansible Agent** عليها.

[**KEY IDEA**] المعمارية «Star» بسيطة: مركز واحد ذكي (Control) يتحكم بنقاط متعددة (Managed) عبر قناة قياسية (SSH/WinRM). لا طبقات وسيطة، لا Coordination Services — لذا الـ Setup والتشخيص سهلان.

[**WATCH OUT**] الـ Ansible + الـ Playbooks + الـ Inventory كلها تعيش على الـ Control Node. الـ Managed Nodes تعرف شيئًا واحدًا فقط: «وصلني أمر SSH — أنقذه».

EXAM 📌 «ما مكونات Ansible Architecture؟» → **Control Node** (Ansible + Playbooks + Inventory — منه التشغيل) و **Managed Nodes** (الـ Targets — تستقبل عبر SSH/WinRM بلا Agent). و«ما أشكال الـ Control Node؟» → Laptop / Linux Server / WSL.

🔗 لكن هل تستحق كل هذا؟ مقارنة صريحة على 3 Servers → **الفصل 5: Comparison**

Servers Comparison مقارنة طرق إدارة ال —

لنفترض أن لديك 3 Linux Servers — Ubuntu, CentOS, Alpine — وتريد تثبيت Nginx عليها جميعًا. ثلاث طرق:

وجه المقارنة	Manual 🖱️	Bash Scripts 📄	Ansible ▶️
آلية العمل	الدخول عبر SSH لكل Server وتنفيذ الأوامر	كتابة Bash Script يتعرف على ال OS وينفذ الأوامر	كتابة YAML يحدد ال Desired State
السرعة والجهد	بطيئة وتكثر فيها الأخطاء	تحتاج إلى Scripting وخبرة أعلى	سريعة، مركزية، Simple Syntax
Idempotency	غير متوفرة تلقائيًا	غير متوفرة تلقائيًا	مدعومة من خلال Ansible Modules
اختلاف ال OS	تحتاج معرفة أوامر كل Distribution	تحتاج logic if/else لكل OS	يتم التعامل مع الاختلافات من خلال Modules

◀ TASK [قراءة الجدول على الواقع] practical lens

لنفس المهمة (تثبيت Nginx على الثلاثة): ال 3 = Manual جلسات SSH و3 صيغ أوامر مختلفة (apt / yum / apk) وتحقق يدوي. ال Bash = Script واحد لكن بفروع لكل Package Manager وكل توزيعية جديدة تضيف فرعًا. ال Task = Ansible واحدة بـ YAML تعمل على الثلاثة — وال Modules تختفي اختلافات التوزيعات، وال Idempotency مدمجة تلقائيًا.

[ASIDE • المفارقة] Bash Scripts خطوة تقدم عن ال Manual — لكنها دين تقني يتراكم: كل توزيعية = فرع، كل تغيير = مراجعة. Ansible يقلب السؤال: لا «كيف أنفذ على كل نظام؟» بل «ما ال State التي أريدها؟» — وال Modules تعرف الباقي.

🎓 EXAM أربعة أزواج تحفظها: Manual = بطء + أخطاء + بلا Bash · Idempotency = مركزي لكن Scripting ثقيل + = Ansible · if/else. YAML مركزي + Idempotency عبر Modules + تجاهل اختلافات ال OS تلقائيًا.

🔗 أين تُستخدم فعليًا؟ ثلاثة ميادين + نقاش Terraform → [الفصل 6: Use Cases](#)

Use Cases مجالات الاستخدام —

5.1 Configuration Management — الأشهر

يمكن استخدامه في:

إدارة الـ Services

ضبط الـ Configuration Files

تحديث الـ Packages

تثبيت الـ Packages

تحديث عدة Servers من مكان واحد

5.2 Infrastructure Provisioning

يمكن لـ Ansible التعامل مع Infrastructure و Cloud APIs لإنشاء أو حذف موارد: **S3 · EC2 · Containers**.

[Ansible و ASIDE · Terraform] في كثير من البيئات يتم الفصل بين مسؤوليات الأدوات:

Ansible

Configuration Management



Infrastructure Configured

Terraform

Infrastructure Provisioning



Infrastructure Created

يُفضل عادةً استخدام **Terraform** للـ Provisioning، و **Ansible** للـ Configuration Management.

5.3 Network Automation

يمكن استخدام Ansible في أتمتة وإدارة: **Firewalls · Switches · Routers** — برمجياً بدل الدخول اليدوي لكل جهاز.

EXAM 🎓 ثلاثة مجالات: **Provisioning** (packages/files/services/multi-server) · **Config Management** (containers/EC2/S3) — ونقاش (Terraform) · **Network Automation** (routers/switches/firewalls). وقاعدة الفصل: Terraform يبني → Ansible يهيئ.

🔗 المجالات واضحة — نُجهز العدة الآن: → **الفصل 7: Installation**

Installation + VS Code

يتم تثبيت Ansible على الـ **Control Node** — اختر طريقك حسب نظامك:

Ubuntu / Debian

```
$ sudo apt update && sudo apt install ansible -y
```

macOS

```
$ brew install ansible
```

Windows

استخدم **WSL** وتشغيل Ubuntu عليه، ثم ثبّت Ansible داخل بيئة الـ Linux.

Python Pip

```
$ pip3 install ansible
```

TASK [المفيدة VS Code Extensions] editor setup

Ansible Extension

YAML Extension

هذه الـ Extensions تساعد في:

Syntax اكتشاف أخطاء الـ

Ansible Editing

YAML Editing

Syntax Highlighting

دقيقتان في تجهيز المحرر تكتشفان أخطاء الـ Indentation قبل أن تصل إلى الـ Run.

[WATCH OUT] التثبيت للـ **Control Node فقط** — الـ Managed Nodes تستقبل عبر SSH/WinRM ولا تحتاج Ansible إطلاقاً.

EXAM طرق التثبيت: Extensions (Windows) · WSL (Any OS) · pip3 (Ubuntu/Debian) · apt (macOS) · brew (المقترحة: YAML) + Ansible من Red Hat.

الأداة جاهزة — الآن نؤمن الطريق إلى الـ Servers: → **الفصل 8: SSH Passwordless**

— Passwordless Setup SSH Keys

في Linux يستخدم Ansible عادةً SSH للتواصل مع ال Managed Nodes. الفكرة:

[The Key Flow - من ينقل ماذا]

Managed Node

يحمل ال Public Key
في Authorized Keys

► — SSH Login — ◀

Control Node

يحمل ال Private Key

8.1 إنشاء ال SSH Key Pair — على ال Control Node

```
$ ssh-keygen -t rsa -b 4096
```

ينتج عن ذلك ملفان:

id_rsa.pub — Public Key

يتم وضعه على ال Managed Node. قفله لا يفتح شيئًا.

id_rsa — Private Key

يُحفظ بشكل سري على ال Control Node — لا يُشارك أبدًا.

8.2 نسخ ال Public Key إلى ال Managed Node

```
$ ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@<MANAGED_NODE_IP>
```

8.3 اختبار الاتصال

```
$ ssh ubuntu@<MANAGED_NODE_IP>
Welcome to Ubuntu 22.04 LTS ... # no password ✓
$ exit
```

إذا تم إعداد ال SSH Keys بشكل صحيح، يمكنك الاتصال بدون إدخال ال Password في كل مرة.

[WATCH OUT] ال Private لا يسافر أبدًا — ال Public هو الوحيد الذي يُنسخ. عكسهما = اتصال لا يعمل + خطر أمني.

EXAM  السلسلة: `ssh-copy-id -i ~/.ssh/id_rsa.pub user@ip` ← `ssh-keygen -t rsa -b 4096` ← اختبار `ssh user@ip`.
النتيجة: أتمتة صامتة بلا Passwords.

🔗 الاتصال مؤتمن — الآن وحدات التنفيذ: الفصل 9: Modules

Modules وحدات التنفيذ —

تعتمد الـ Ansible Tasks على الـ **Modules** — والـ Module هو جزء جاهز من **Ansible** يقوم بوظيفة محددة. أنت تستدعيه وتعطيه الـ State، وهو يعرف الـ How.

TASK [جدول الـ Modules] core modules

Module	الاستخدام
apt	إدارة Packages على Debian/Ubuntu
yum	إدارة Packages على الأنظمة التي تستخدم yum
ping	اختبار الاتصال
service	إدارة Services
copy	نسخ Files
command	تنفيذ Commands
shell	تنفيذ Shell Commands

TASK [مثال عملي] module in action

```
- name: Install Nginx
  apt:
    name: nginx
    state: present
```

Package Management ▼

Ansible Module ▼

apt

[KEY IDEA] الـ Module يلخّص الـ Platform-specific details: نفس الـ Intent («nginx موجود») يُنفَّذ بالـ Command الصحيح لكل توزيع — أنت لا تكتب apt-get ولا yum install يدويًا.

[WATCH OUT] command و shell — ينفّذان أوامر خام — استخدامهما بكثرة يُفقدك الـ Idempotency والأناقة. ابحث أولاً عن Module مخصص لمهمتك.

EXAM 🎓 احفظ السبعة: shell · command · copy · service · ping · yum (yum-based) · apt (Debian/Ubuntu) — ووظيفة كل واحد حرفيًا من الجدول.

🔗 العدة جاهزة — نحتاج قائمة الأهداف: → **الفصل 10: Inventory**

Servers Inventory ملف ال —

ال **Inventory** هو الملف الذي يخبر **Ansible** بال **machines** التي يستطيع إدارتها. يمكن أن يحتوي على: **IP Addresses** · **Hostnames** · **Groups** · **Connection Variables**.

TASK [مثال hosts.ini] inventory example

```
[web]
192.168.1.101 ansible_user=ubuntu
192.168.1.102 ansible_user=ubuntu

[db]
192.168.1.201 ansible_user=ubuntu
```

TASK [قراءة ال Groups] group trees

Group: db

```
db
└─ 192.168.1.201
```

Group: web

```
web
├─ 192.168.1.101
└─ 192.168.1.102
```

[**DEBUG** · التشرح] [web] = اسم Group بين أقواس · الأسطر تحته = الأعضاء · `ansible_user=ubuntu` = Connection Variable
تحدد ال SSH User — تُعرّف مرة بدل تكرارها.

[**KEY IDEA**] ال Inventory = «مصدر الحقيقة» لخريطة ال Infrastructure: Server جديد = سطر. User متغير = تعديل واحد. كل ال
Commands وال Playbooks القادمة تحيل إليه بـ `-i hosts.ini`.

[**WATCH OUT**] أسماء ال Groups حساسة: [webs] ≠ [web]. خطأ حرف في الاستهداف = «لا Hosts مطابقة» بلا تنبيه لل Typo.

EXAM ال Inventory يحتوي: **IPs · Hostnames · Groups · Connection Variables** — ال Groups وحدات استهداف، ويُمرّر الملف بـ `-i`.

ال Modules + ال Inventory جاهزان — أول تنفيذ فوري: → **الفصل 11: Ad-hoc**

Ad-hoc Commands ⚡

Ad-hoc Commands تُستخدم عندما تريد تنفيذ Task سريعة بدون إنشاء Playbook كامل — هذا هو الجانب Imperative.

TASK [الصيغة العامة] general syntax

```
ansible <group_or_all> -i <inventory_file> -m <module_name> -a "<module_arguments>"
```

TASK [الأمثلة الثلاثة] practical examples

11.1 اختبار الاتصال:

```
$ ansible all -i hosts.ini -m ping
192.168.1.101 | SUCCESS => { "ping": "pong" }
192.168.1.201 | SUCCESS => { "ping": "pong" }
```

11.2 معرفة ال Uptime:

```
$ ansible web -i hosts.ini -a "uptime"
```

11.3 تثبيت Nginx:

```
$ ansible all -i hosts.ini -m apt -a "name=nginx state=present" --become
```

TASK [معنى أهم ال Options] flags decoded

become--

تشغيل العملية بصلاحيات مرتفعة

a-

Module Arguments

m-

اختيار ال Module

[KEY IDEA] اقرأ أي Ad-hoc Command كجملة: «على المجموعة X، وبال Y Inventory، استخدم Module Z بال W Arguments، بصلاحيات مرتفعة» — كل Flag جزء من الجملة.

[WATCH OUT] عند حذف -m يُستخدم ال Module command افتراضياً — لهذا -a "uptime" تعمل مباشرة. و Ad-hoc لمهام لا تتكرر — المتكرر يذهب لـ Playbook.

EXAM 📝 الهيكل: --become ← -a ← -m ← -i ← Target. والافتراضي عند غياب -m هو command.

🔗 السيناريوهات الكاملة تحتاج لغة ملفات — **الفصل 12: YAML**

YAML Basics أساسيات الصيغة —

يعتمد الـ YAML بشكل كبير على الـ **Indentation** — الـ structure كله يُقرأ من المسافات.

TASK [قواعد مهمة] ground rules

الـ Indentation

استخدم **Spaces** وليس **Tabs** — ويُفضل 2 **Spaces**.

بداية الـ YAML Document

```
---
```

TASK [هياكل الـ Data] data structures

③ Dictionary / Mapping

```
server:
  name: web01
  environment: production
```

② List

```
packages:
- nginx
- git
- curl
```

① Key-Value

```
name: nginx
state: present
```

[**KEY IDEA**] ثلاثة هياكل فقط — **Dictionary · List · Key-Value** — وكل Playbook ستراه لاحقًا هو تركيبة منهما: List من Dictionaries. إتقان هذه الصفحة = قراءة أي Playbook بلا حيرة.

[**WATCH OUT**] الـ Tab داخل ملف YAML = فشل الـ Parsing غالبًا بـ Error بعيد عن سطره الحقيقي. اضبط الـ Editor على «Spaces = 2» قبل كتابة أول سطر.

EXAM القواعد: ابدأ بـ **Spaces** · --- لا **Tabs** (يفضل 2) · الهياكل: **Key-Value, List, Dictionary/Mapping**.

اللغة جاهزة — أول Playbook كامل: → **الفصل 13**

— Playbooks الكتيبات

الـ **Playbook** هو ملف YAML يحتوي على مجموعة من الـ **Plays**. كل **Play** تستهدف مجموعة من الـ **hosts** وتحتوي على مجموعة من الـ **Tasks**.

◀ TASK [الـ Hierarchy الأساسية] the structure

```

Playbook
├── Play
│   ├── Task
│   ├── Task
│   └── Task
└── Play
    ├── Task
    └── Task
  
```

◀ TASK [مثال: تثبيت وتشغيل Apache] أنشئ ملف site.yml

```

---
- name: Install and Start Apache Web Server
  hosts: web
  become: yes

  tasks:

    - name: Install Apache Package
      apt:
        name: apache2
        state: present

    - name: Ensure Apache Service is Running and Enabled
      service:
        name: apache2
        state: started
        enabled: yes
  
```

[**DEBUG** • القراءة السريعة] الملف = List فيها Play واحد • الـ Play تستهدف `hosts: web` (من الـ Inventory) • `become: yes` = صلاحيات Task1 • root تضمن الحزمة Task2 • `present` تضمن الـ `started + enabled` Service.

[**KEY IDEA**] لاحظ أن كل **Name = Task + وصفي + State + Module** — أنت تقرأ الـ Playbook كقصة: «تثبيت Apache، ثم تأكد أن الخدمة تعمل ومفعلة». الـ YAML يجعل الـ Automation مقروءًا كوثيقة.

📌 **EXAM** الـ **hosts targeting** = **Play** → **Tasks** → **Plays** → **Playbook** + قائمة **Tasks** • وكل Task يستدعي Module بـ **State** محددة.

🔗 الكُتَيْب مكتوب — الآن التشغيل وفهم النتائج: → **الفصل 13-b**

Running + Results التشغيل والنتائج

execute it [TASK تشغيل الـ Playbook]

```
$ ansible-playbook -i hosts.ini site.yml
```

الـ Command يتكون منطقياً من:

Playbook `site.yml` شغل هذا

Inventory `hosts.ini` استخدم هذا

`ansible-playbook`

the three statuses [TASK فهم نتائج Ansible]

failed

الـ Task فشلت بسبب **Error**.

changed

Ansible قام بإجراء تغيير بنجاح.

ok

الـ host بالفعل في الـ **Desired State** —
لم يكن هناك تغيير مطلوب.

[The status flow]

branches

Task

failed

Error

changed

Changed

ok

No change

[**KEY IDEA**] كل Task تنتهي بأحد ثلاثة مصائر — والفرق بين **ok** و **changed** هو جوهر الـ Idempotency: نفس الـ Playbook مرة ثانية = كل شيء **ok** وصفر **Changes**.

[**WATCH OUT**] عند **failed**: اسم الـ Task الأحمر يخبرك أي خطوة، والسطر أسفله يخبرك لماذا — اقرأهما قبل أي بحث خارجي.

🎓 **EXAM** التشغيل: `ansible-playbook -i hosts.ini site.yml` · الحالات: **ok** = بالفعل بالحالة المطلوبة · **changed** = تغيير ناجح · **failed** = Error وتوقف.

والآن.. الـ Playbook كبر. مئات الـ Tasks في ملف واحد؟ الحل: → **الفصل 14: Roles**

Roles الفكرة ولماذا نستخدمها —

عندما يكبر الـ **Playbook**، قد يحتوي على مئات الـ **Files** · **Templates** · **Handlers** · **Variables** · **Tasks**. وضع كل شيء في ملف واحد يجعل الـ project صعب التنظيم والـ **Maintenance**. هنا يأتي دور **Ansible Roles**.

KEY IDEA · ما هو الـ **Role** ؟ [الـ **Role** هي طريقة منظمة لتقسيم الـ Automation Code إلى مكونات **Modular & Reusable** — كل **Role** مسؤولة عن وظيفة محددة:

Application

- └─ nginx Role
- └─ mysql Role
- └─ docker Role

بدل أن يكون لديك **Playbook** ضخيم يحتوي على كل التفاصيل.

TASK [لماذا نستخدم Roles؟] the benefits

مشاركة الـ Code مع الـ Team

سهولة الـ Maintenance

Reusability

Modularity

Organization

إعادة الاستخدام في عدة Projects

TASK [بدون Roles مقابل Roles مع] the killer comparison

Roles مع ✓

```

graph LR
    NginxRole[Nginx Role] --- ProjectA[Project A]
    NginxRole --- ProjectB[Project B]
    NginxRole --- ProjectC[Project C]
  
```

Roles بدون ✗

```

Project A → Rewrite Nginx Setup
Project B → Rewrite Nginx Setup
Project C → Rewrite Nginx Setup
  
```

Reuse Everywhere

Reusable Role

Build Once

EXAM تعريف الـ **Role**: طريقة منظمة لتقسيم الـ **Automation** إلى مكونات **Modular & Reusable** — كل **Role** مسؤولة عن وظيفة محددة. والفائدة المختصرة: **Build Once** → **Reuse Everywhere**.

الفكرة واضحة — ماذا يوجد داخل الـ **Role** فعلياً؟ هيكل الـ **Directories**: → **الفصل 15**

1/2 — tasks & handlers Role Structure

عند إنشاء Role، يتم تنظيمها في **Directories** لها وظائف محددة:

```
nginx/
├─ tasks/
├─ handlers/
├─ vars/
├─ defaults/
├─ files/
├─ templates/
└─ meta/
```

19.1 tasks / — المكان الأساسي

هذا هو المكان الذي تضع فيه الـ **Tasks** التي سينفذها الـ Role. أمثلة:

Start service

Configure packages

Create directories

Install Nginx

19.2 handlers / — المهام المشروطة

يحتوي على الـ **Handlers**. الـ Handler هو Task يتم تنفيذها عندما يتم عمل **Trigger** لها — غالبًا من خلال `notify`:

[The handler flow - المثال الشائع]

Restart / Reload Service

▼ Handler

▼ notify

▼ Configuration File Changed

إذا تغيّر Nginx Configuration File، يمكن تشغيل Handler لعمل Restart أو Reload للـ Nginx Service.

[**KEY IDEA**] الـ Handler = **رد فعل مشروط** لا خطوة دائمة: يتلقى Trigger من `notify` عند حدوث تغيير فعلي — فلا Restarts عشوائية في كل Run، ولا تغييرات تبقى بلا تأثير.

[**WATCH OUT**] الـ `notify` يطابق اسم الـ Handler **حرفيًا** — خطأ إملائي = الـ Handler لن يشتغل أبدًا ولا يظهر Error.

EXAM `/tasks` = الـ Tasks الرئيسية للـ Tasks `handlers/` · Role تُطلق بـ `notify` عند التغيير — المثال النموذجي: Config File Changed → notify → Restart Service

باقي المجلدات: vars, defaults, files, templates, meta → **الفصل 15b**

2/2 — vars إلى meta Role Structure

19.3vars / — المتغيرات الثابتة

يحتوي على الـ **Variables** الخاصة بالـ Role — ولها **High Precedence** مقارنةً بمصادر الـ Variables الأخرى. تُستخدم للقيم الخاصة بالـ Role التي لا يُفترض تغييرها بسهولة.

19.4defaults / — المتغيرات القابلة للتجاوز

يحتوي على الـ **Default Variables** — مصممة بحيث يمكن عمل **Override** لها بسهولة عند استخدام الـ Role:

Can be overridden ▼

Default Values ▼

/defaults

19.5files / — الملفات الثابتة

يحتوي على الـ **Static Files** التي يمكن للـ Role نسخها إلى الـ Managed Nodes:

```
files/
└─ index.html
```

19.6templates / — القوالب الديناميكية

يحتوي على ملفات **Jinja2 Templates** — غالبًا بامتداد `.j2`. تُستخدم لإنشاء Configuration Files بشكل **Dynamic** اعتمادًا على الـ Variables:

Dynamic Configuration File ▼

Jinja2 Template +

Variables

19.7meta / — البيانات الوصفية

يحتوي على الـ **Metadata** الخاصة بالـ Role — ومن ضمن استخداماته تعريف الـ **Dependencies**: مثل Role أخرى يجب تنفيذها لكي يعمل الـ Role الحالي بشكل صحيح.

[WATCH OUT] أكثر خلط شائع: **/vars** = قيم داخلية عالية الـ Precedence (لا تُلمس) · **/defaults** = قيم مصممة للـ Override من المستخدم. الخاطئ في المكان = Role لا تقبل التخصيص أو تقبله أكثر من اللازم.

📌 **EXAM** السبعة بوظائفهم: **tasks** (التنفيذ) · **handlers** (مشروطة بـ notify) · **vars** (ثابتة عالية الأولوية) · **defaults** (قابلة للـ Override) · **templates** (Static) · **files** (Jinja2 ديناميكية) · **meta** (Metadata + Dependencies).

🔗 الهيكل مكتمل — كيف نستدعي الـ Role؟ وما منصة Galaxy؟ → [الفصل 16](#)

Galaxy + Using a Role

TASK [استخدام Role داخل Playbook] calling a role

بدل وضع كل ال Tasks مباشرة داخل ال Playbook، يمكن استدعاء ال Role:

```
---
- name: Configure Web Server
  hosts: web

  roles:
    - nginx
```

هنا ال Playbook أصبح مسؤولاً عن ال **high-level orchestration**، بينما ال Role تحتوي على تفاصيل ال **implementation**:

```
Playbook
├── nginx Role
│   ├── tasks/
│   ├── handlers/
│   ├── templates/
│   ├── files/
│   └── variables
```

TASK [منصة Ansible Galaxy] the ecosystem

Ansible Galaxy هي منصة و **ecosystem** لاكتشاف ومشاركة وتثبيت Ansible Content. يمكن العثور عليها: **Roles** و **Collections**.

[ASIDE • تشبيه مبسط] Galaxy بالنسبة للـ Ansible Content مثل:

Ansible Galaxy



Roles / Collections

pip



Python Packages

npm



Node.js Packages

الفائدة الأساسية هي **Reusability** — بدل أن تبدأ كل Role من الصفر، تجد Content جاهزًا تستخدمه أو تشاركه في ال Project.

[**DEBUG** • فصل المسؤوليات] ال Playbook يقول **ماذا** يحدث (orchestration) — وال Role تحمل **كيف** يحدث بالتفصيل (implementation). هذا الفصل يجعل ال Playbook الرئيسي قصيرًا مقروءًا مهما كبر المشروع.

EXAM 🎓 استدعاء Role داخل `roles: - nginx` Playbook: — وال Playbook = orchestration · ال Role = implementation. Galaxy = منصة اكتشاف ومشاركة Roles و Collections (شبيهة npm/pip).

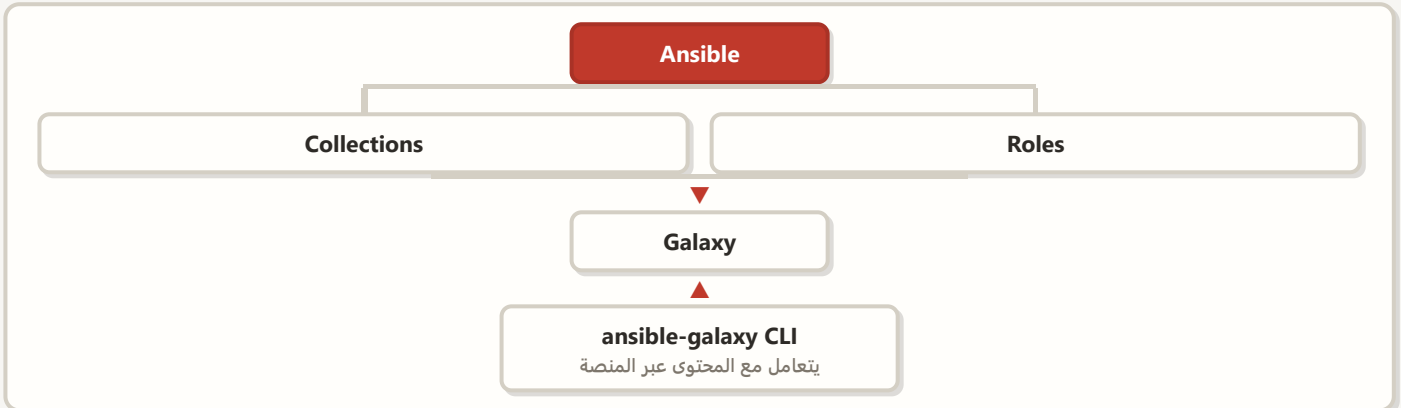
🔗 **الفرق بين Role و Galaxy و ansible-galaxy؟ → الفصل 17**

ansible-galaxy Role vs Galaxy vs

من المهم عدم الخلط بين الثلاثة — كل واحد منهم شيء مختلف:

Concept	معناه
Role	وحدة Ansible Automation قابلة لإعادة الاستخدام
Galaxy	Platform / Ecosystem لاكتشاف ومشاركة Ansible Content
ansible-galaxy	CLI للتعامل مع Ansible Content

TASK [علاقتهم ببعض] the relationship



TASK [تثبيت Role باستخدام ansible-galaxy] install

```
$ ansible-galaxy install username.role_name
```

Role available in your project ▼

ansible-galaxy install

Find Role ▼

Galaxy

TASK [إنشاء Role جديدة] init

```
$ ansible-galaxy init my_custom_role
# تلقائياً standard structure ينشئ الـ:
# tasks/ handlers/ vars/ defaults/ files/ templates/ meta/
```

بدل أن تقوم بإنشاء كل Directory يدوياً.

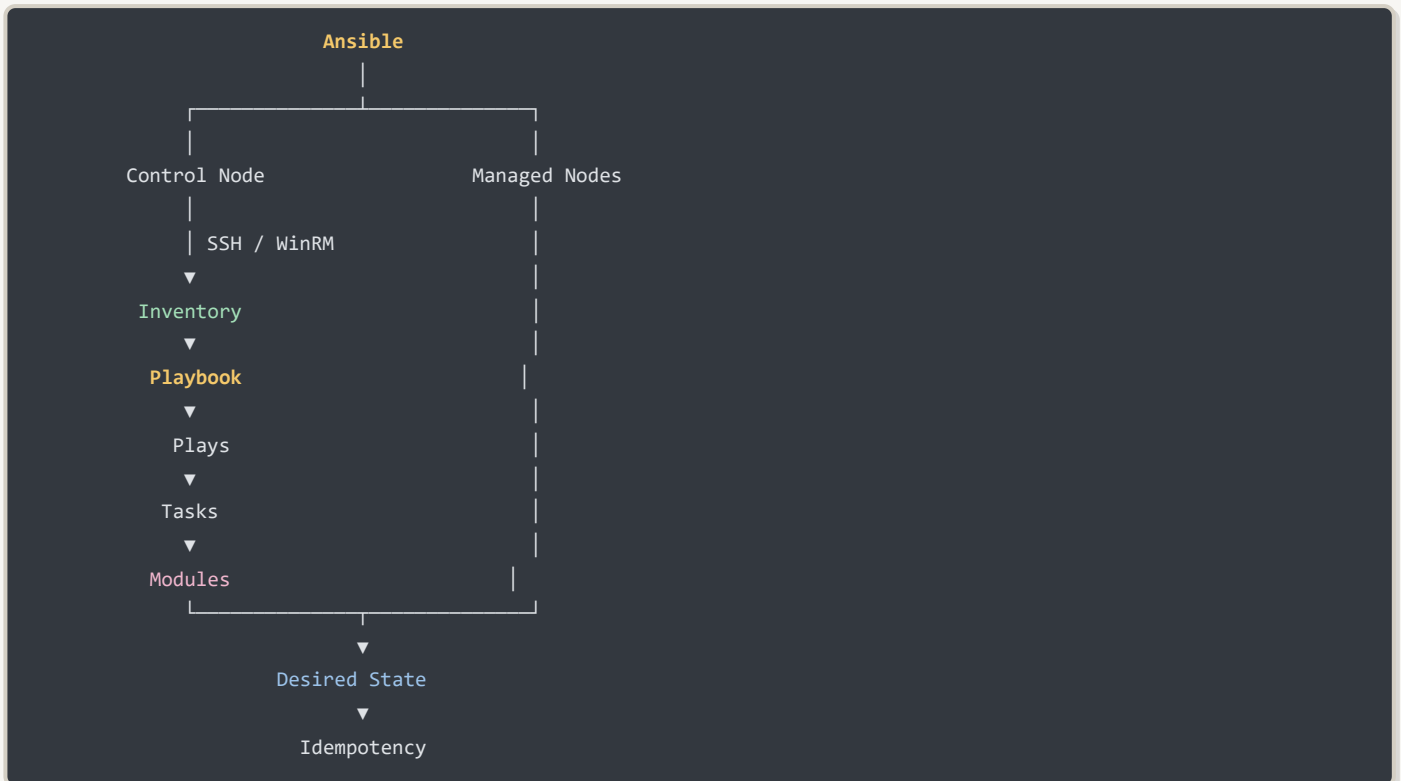
[KEY IDEA] احفظ الثلاثية: الـ **Role** = المحتوى نفسه · **Galaxy** = السوق/المنصة · **ansible-galaxy** = الأداة (CLI) التي تجلب وتنشئ المحتوى. سؤال المقابلة يكشف من يخلط بينهم.

EXAM أمان فقط: `ansible-galaxy install username.role_name` (تثبيت) · `ansible-galaxy init my_custom_role` (إنشاء) هيكلي قياسي كامل تلقائياً.

كل القطع جاهزة — الآن نجمعها في صورة واحدة: → **الفصل 18: الصورة الكاملة**

— الصورة الكاملة The Complete Picture 📖

كل ال Concepts التي تعلمناها ترتبط ببعضها:



scaling up [Project] TASK [وعندما يكبر ال Project]

```

Playbook
├── Role: nginx (tasks/ handlers/ templates/ files/ defaults/ vars/ meta/)
├── Role: docker
└── Role: mysql
  
```

galaxy enters [TASK] [وعندما تريد Reusable Content جاهز]

```

Ansible Galaxy
├── Find Roles
├── Find Collections
└── Reuse Ansible Content
  
```

KEY IDEA [اقرأ ال Picture من الأعلى للأسفل مرة واحدة: **Control** يستخدم **Inventory** ليوجه **Playbook** → **Plays** → **Tasks** → **Modules** → **Desired State** يحميها ال **Idempotency**. وعند التوسع: **Roles** تنظم، و **Galaxy** تزود. هذه خريطة Ansible كلها.

DEBUG • كيف تراجع بها؟ [غطّ أي جزء وأسأل نفسك: «ماذا يأتي بعد ال Playbook؟» — إن أجبت دون تردد، أنت تفهم ال Machine لا تحفظ مربعاتها.

EXAM 🎓 سلسلة التنفيذ الكاملة بالترتيب: **Inventory** → **Playbook** → **Plays** → **Tasks** → **Modules** → **Desired State** → **Idempotency** — وعند التوسع: **Roles** لكل وظيفة + **Galaxy** للمحتوى الجاهز.

تبقي شيء واحد: سلسلة الحفظ وال Revision النهائي → **الفصل الأخير**

Mental Model — سلسلة الحفظ

لو هتخفظ سلسلة واحدة فقط، خليك مع دي — كل Concept يجيب على سؤال:

يجيب على سؤال...	Concept
"Which machines?" — أي الأجهزة؟	Inventory
"What should happen?" — ماذا يجب أن يحدث؟	Playbook
"How do I organize and reuse the automation?"	Role
"One specific unit of work" — وحدة عمل محددة	Task
"What Ansible capability performs the operation?"	Module
"Where can I find/share reusable Ansible content?"	Galaxy

TASK [وبشكل أبسط] the chain

Desired State

Managed Node

Module

Task

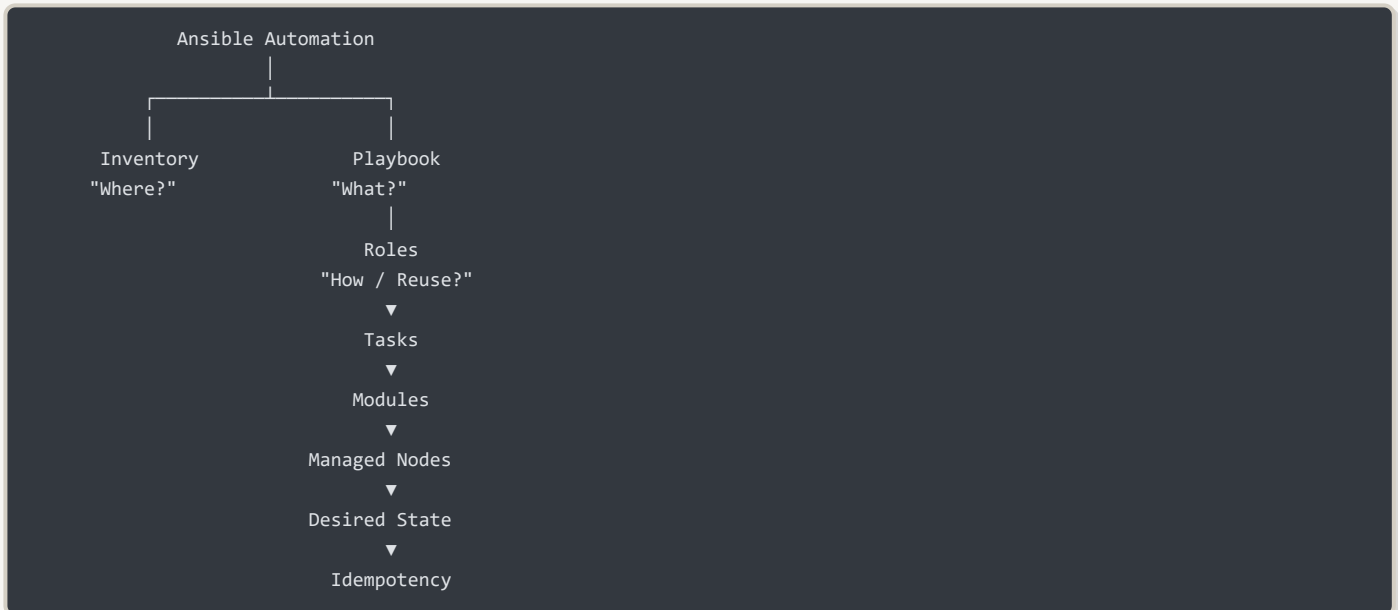
Role

Playbook

YOU

[ASIDE • طريقة الاستخدام] احفظ السلسلة كسؤال وجواب: في ال Interview لو سئلت «ما وظيفة ال Role؟» — الجواب هو السؤال الذي تجيبه: "How do I organize and reuse the automation?" من يربط ال Concept بسؤاله، يفهمه ولا ينساه.

TASK [Final Mental Model] the last chain



[KEY IDEA] السلسلة من الأعلى للأسفل = رحلة ال Automation كاملة: أنت تكتب **Playbook** ينظمها **Role** بخطواتها **Tasks** التي تنفذها **Modules** على ال **Managed Nodes** حتى تصل للـ **Desired State**.

▶ آخر صفحة: جدول ال Quick Revision الشامل — [التالي](#)

جدول المراجعة السريعة + الخلاصة

16 مصطلحًا في جدول واحد — آخر شيء تراجعته قبل ال Interview:

Term	الفكرة الأساسية
Ansible	Automation Tool
Control Node	الجهاز الذي يشغل Ansible
Managed Node	الجهاز الذي يتم إدارته
Inventory	يحدد ال Hosts وال Groups
Module	وظيفة جاهزة لتنفيذ Operation معينة
Task	وحدة عمل واحدة
Play	مجموعة Tasks تستهدف Hosts
Playbook	ملف YAML يحتوي على Plays
Role	طريقة Modular & Reusable لتنظيم Automation
Handler	Task يتم Trigger لها عند حدوث تغيير
Template	Dynamic file باستخدام Jinja2
Galaxy	Platform / Ecosystem لمشاركة واكتشاف Ansible Content
ansible-galaxy	CLI للتعامل مع Ansible Content
Idempotency	تكرار التنفيذ لا يسبب تغييرات غير ضرورية
Desired State	الحالة التي تريد أن يكون عليها النظام
Agentless	لا تحتاج Ansible Agent على Managed Nodes

TASK [الخلاصة] the summary

Inventory → مين هتدير؟	Playbook → عايز تعمل إيه؟
Role → نَظِّم ال Automation وخليها Reusable	Task → خطوة محددة
Module → الأداة التي تنفذ ال Operation	Galaxy → مكان تلاقى منه Ansible Content جاهز
ansible-galaxy → ال CLI الذي تستخدمه للتعامل مع هذا ال Content	

[KEY IDEA • الختام] Inventory يسأل «مين؟» • Playbook يسأل «ماذا؟» • Role ينظِّم • Task تنفِّذ • Module تُجبر • Galaxy تزوّد.
سلسلة واحدة من ست حلقات = Ansible كله. ►